

Exercise 1: Control Structures

Table Creation:

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    CustomerName VARCHAR2(50),  
    Age NUMBER,  
    Balance NUMBER,  
    IsVIP VARCHAR2(5)  
);  
  
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    InterestRate NUMBER,  
    DueDate DATE,  
    CONSTRAINT fk_customer FOREIGN KEY (CustomerID) REFERENCES Customers  
(CustomerID)  
);  
  
INSERT INTO Customers VALUES (1, 'Rahul', 65, 15000, 'FALSE');  
INSERT INTO Customers VALUES (2, 'Priya', 45, 8000, 'FALSE');  
INSERT INTO Customers VALUES (3, 'Rohan', 70, 12000, 'FALSE');  
INSERT INTO Customers VALUES (4, 'Anita', 30, 5000, 'FALSE');  
  
INSERT INTO Loans VALUES (101, 1, 10, SYSDATE+20);  
INSERT INTO Loans VALUES (102, 2, 8, SYSDATE+50);  
INSERT INTO Loans VALUES (103, 3, 9, SYSDATE+10);  
INSERT INTO Loans VALUES (104, 4, 11, SYSDATE+100);  
  
COMMIT;
```

Scenario-1

```
SET SERVEROUTPUT ON;  
DECLARE  
BEGIN  
    FOR c IN (SELECT CustomerID, Age FROM Customers)  
    LOOP  
        IF c.Age > 60 THEN
```

```

        UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = c.CustomerID;
    END IF;
END LOOP;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Interest rates updated.');
```

END;

/

```

SELECT * FROM Loans;
```

OUTPUT:

The screenshot shows an IDE with three tabs: 'CREATE TABLE Customers (Untitled-1', 'SET SERVEROUTPUT ON; Untitled-2', and 'SET SERVEROUTPUT ON; Untitled-4'. The main editor displays the SQL code from the previous block. Below the editor, the 'QUERY RESULT' tab is active, showing the output of the query. It indicates that 4 rows were fetched in 0.161 seconds. The output is a table with 5 columns: LOANID, CUSTOMERID, INTERESTRATE, and DUE DATE. The data rows are as follows:

	LOANID	CUSTOMERID	INTERESTRATE	DUE DATE
1	101	1	8	20-07-26
2	102	2	8	19-08-26
3	103	3	7	10-07-26
4	104	4	11	08-10-26

The status bar at the bottom shows 'Ln 19, Col 21, Spaces: 4, UTF-8, CR/LF, () PL/SQL, (Dedicated) SYS@localhost'.

Scenario - 2

```

SET SERVEROUTPUT ON;
```

```
BEGIN
    FOR c IN (SELECT CustomerID, Balance FROM Customers)
    LOOP
        IF c.Balance > 10000 THEN
            UPDATE Customers
                SET IsVIP = 'TRUE'
            WHERE CustomerID = c.CustomerID;
        END IF;
    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('VIP status updated.');
```

END;

/

```
SELECT * FROM CUSTOMERS;
```

OUTPUT

CREATE TABLE Customers (
Untitled-1
SET SERVEROUTPUT ON;
Untitled-2
SET SERVEROUTPUT ON;
Untitled-4

```

1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      FOR c IN (SELECT CustomerID, Balance FROM Customers)
5      LOOP
6          IF c.Balance > 10000 THEN
7              UPDATE Customers
8              SET IsVIP = 'TRUE'
9              WHERE CustomerID = c.CustomerID;
10         END IF;
11     END LOOP;
12
13     COMMIT;
14
15     DBMS_OUTPUT.PUT_LINE('VIP status updated.');
```

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
QUERY RESULT
SCRIPT OUTPUT
SQL HISTORY
TASK MONITOR

All rows fetched: 4 in 0.162 seconds

	CUSTOMERID	CUSTOMERNAME	AGE	BALANCE	ISVIP
1	1	Rahul	65	15000	TRUE
2	2	Priya	45	8000	FALSE
3	3	Rohan	70	12000	TRUE
4	4	Anita	30	5000	FALSE

Scenario 3

```

SET SERVEROUTPUT ON;

BEGIN
    FOR l IN (
        SELECT c.CustomerName, l.DueDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.DueDate <= SYSDATE + 30
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Reminder: '
            || l.CustomerName
            || ' has a loan due on '

```

```

        || TO_CHAR(l.DueDate,'DD-MON-YYYY')
    );
END LOOP;
END;
/

```

OUTPUT

SET SERVEROUTPUT ON; Untitled-2

SET SERVEROUTPUT ON; Untitled-4

SET SERVEROUTPUT ON; Untitled-5

1 SET SERVEROUTPUT ON;

2

3 BEGIN

4 FOR l IN (

5 SELECT c.CustomerName, l.DueDate

6 FROM Customers c

7 JOIN Loans l

8 ON c.CustomerID = l.CustomerID

9 WHERE l.DueDate <= SYSDATE + 30

10)

11 LOOP

12 DBMS_OUTPUT.PUT_LINE(

13 'Reminder: '

14 || l.CustomerName

15 || ' has a loan due on '

16 || TO_CHAR(l.DueDate,'DD-MON-YYYY')

17);

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULT

SCRIPT OUTPUT

SQL HISTORY

TASK MONITOR

Reminder: Rahul has a loan due on 20-JUL-2026

Reminder: Rohan has a loan due on 10-JUL-2026

PL/SQL procedure successfully completed.

Reminder: Rahul has a loan due on 20-JUL-2026

Reminder: Rohan has a loan due on 10-JUL-2026